Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mocr functions

Synopsis

The functions prefixed with Mocr make up the Optical Character Recognition module. The Optical Character Recognition module is template-based and reads/verifies mechanically generated, character strings in 8-bit grayscale images, and provides results such as quality scores and validity flags. It is especially designed to operate on character strings with known lengths in degraded images with a few degrees of rotation in the target image. Functions are available to create, save, restore, modify, and inquire about fonts. The module also allows you to calibrate fonts, define search constraints, and read/verify character strings in the target image. The font information can be saved in an Aurora Imaging Library-compatible (.mfo) font file format that can be used with other Aurora Imaging Library software products.

Functions

MocrAllocFont
MocrAllocResult
MocrCalibrateFont
MocrControl
MocrCopyFont
MocrDraw
MocrFree
MocrGetResult
MocrHookFunction
MocrImportFont
MocrInquire
MocrModifyFont
MocrPreprocess
MocrReadString
MocrRestoreFont
MocrSaveFont
MocrSetConstraint
MocrStream
MocrVerifyString

MocrAllocFont

Synopsis

Allocate an OCR font context.

Syntax

```
AIL_ID MocrAllocFont(
    AIL_ID SysId,               //in
    AIL_INT64 FontType,         //in
    AIL_INT CharNumber,         //in
    AIL_INT CharCellSizeX,      //in
    AIL_INT CharCellSizeY,      //in
    AIL_INT CharOffsetX,        //in
    AIL_INT CharOffsetY,        //in
    AIL_INT CharSizeX,          //in
    AIL_INT CharSizeY,          //in
    AIL_INT CharThickness,      //in
    AIL_INT StringLength,       //in
    AIL_INT64 InitFlag,         //in
    AIL_ID *FontContextOcrIdPtr //out
)
```

Description

This function allocates an OCR font context on the specified system. An OCR font context contains the OCR font, target image information, and processing controls.

If the OCR font context already exists as an OCR font file that matches your constraints and character sizes, such as *SEMI_M12-92.mfo* (SEMI M12-92) or *SEMI_M13-88.mfo* (SEMI M13-88), restore it using MocrRestoreFont(). If you need a modified version of a SEMI font (changing its character size, offset, thickness, or the number of characters in the OCR font context) or need a user-defined font, you should allocate a new OCR font context. Note that using either M_SEMI_M12_92 or M_SEMI_M13_88 automatically sets the OCR font context type to M_CONSTRAINED.

A newly allocated OCR font context must have its grayscale character representations initialized using MocrImportFont() or MocrCopyFont(). Each part of the OCR font context can be changed using MocrControl() and MocrModifyFont(). Constraints are set using MocrSetConstraint().

When allocating an OCR font context, you must specify an OCR font context type. This can be modified later using MocrControl() with M_CONTEXT_CONVERT. If your target image has a visible threshold difference between the target characters and the background, uniform illumination, and unknown spacing between characters, and unknown string lengths, and if your font is proportional, start by allocating an M_GENERAL OCR font context. If your target image contains noisy, scratched, or low contrast characters, or if the illumination is not uniform, or if your font includes broken characters start by allocating an M_CONSTRAINED OCR font context.

You must calibrate the OCR font context when the cell, offset, size, or thickness of the characters in your target image differs from the OCR font. Font calibration can be either automatic (using MocrCalibrateFont()) or manual (using MocrControl()). Note, to use MocrCalibrateFont(), you must use an M_CONSTRAINED OCR font context.

If you change any controls or constraints, use MocrPreprocess() to speed up any following read or verify operation.

If you intend to reuse a font that you modified, save it using MocrSaveFont(), and restore it using MocrRestoreFont() when you need it.

When the OCR font context is no longer required, release it using MocrFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the OCR font context's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the OCR font context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

FontType INQ

Specifies the OCR font context type and the type of its font. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the context type and the type of its font																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_USER_DEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEMI_M12_92	Specifies a font respecting the SEMI M12-92 standard as the type of font. StringLength must be set to 12, the constraints (MocrSetConstraint()) for the target image are preset, and a checksum calculation is activated.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEMI_M13_88	Specifies a font respecting the SEMI M13-88 standard as the type of font. StringLength must be set to 18, the constraints (MocrSetConstraint()) for the target image are preset, and a checksum calculation is activated.	c		e	f	g	a		i		j	k	l	m	n	b

M_USER_DEFINED +	Specifies a general, user-defined type of font. No maximum string length is set, the scale is set to 1.0, and the character spacing is the same as CharCellSizeX. No checksum is automatically associated with this OCR font context. You must specify a combination value from the following table: • To set the OCR font context type	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_USER_DEFINED.

You must add one of the following values to the above-mentioned value to set the OCR font context type.

For more information about which context type to use, see the Guidelines for choosing context types section of Chapter 13: Optical character recognition.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_USER_DEFINED to specify the context type INQ																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CONSTRAINED	Specifies an OCR font context that works well with degraded target images and requires more information about the target string, but provides a more robust search. Note that using either M_SEMI_M12_92 or M_SEMI_M13_88 automatically sets the OCR font context type to M_CONSTRAINED. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies an OCR font context that works well with clean target images. It requires less information about the target string, but provides a less robust search. If you are using clean target images but they are complex, have lighting variations, or require better binarization, you might consider trying the Aurora Imaging Library String Reader module, which is more suitable for these cases. Note that under the correct conditions, OCR is typically faster.	c		e	f	g	a		i		j	k	l	m	n	b

CharNumber INQ

Specifies how many characters can be stored in the OCR font context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of characters to store																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

0 < *Value* <= 256	Specifies the maximum amount of characters that can be stored. At most 256 characters can be stored.	c		e	f	g	a		i		j	k	l	m	n	b

CharCellSizeX INQ

Specifies the width of the characters in the OCR font context, in pixels.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the width of the characters in the context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
6 <= *Value* <= 256	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

CharCellSizeY INQ

Specifies the height of the characters in the OCR font context, in pixels.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the height of the characters in the context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
6 <= *Value* <= 256	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

CharOffsetX INQ

Specifies the distance between the edge of each character and their surrounding cell, along the X-axis, in pixels.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the X-character offset																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
1 <= *Value* <= 250	Specifies the horizontal distance, in pixels. Typically, this is set to the same value as the character thickness. > Note that the minimum distance is 1 pixel.	c		e	f	g	a		i		j	k	l	m	n	b

CharOffsetY INQ

Specifies the distance between the edge of each character and their surrounding cell, along the Y-axis, in pixels.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Y-character offset

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
1 <= *Value* <= 250	Specifies the vertical distance, in pixels. > Note that the minimum distance is 1 pixel.	c		e	f	g	a		i		j	k	l	m	n	b

CharSizeX INQ

Specifies the width of the widest character in the font without its surrounding cell, in pixels.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the width of the widest character in the font without its surrounding cell																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
6 <= *Value* <= 256	Specifies the width, in pixels. This should be equal to: CharCellSizeX - (CharOffsetX *2). > Note that, to be able to search for a string over a range of angles, the font context's character size must be greater than 16x16.	c		e	f	g	a		i		j	k	l	m	n	b

CharSizeY INQ

Specifies the height of the tallest character in the font without its surrounding cell, in pixels.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the height of the tallest character in the font without its surrounding cell																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
6 <= *Value* <= 256	Specifies the height, in pixels. This should be equal to: CharCellSizeY - (CharOffsetY *2). > Note that, to be able to search for a string over a range of angles, the font context's character size must be greater than 16x16.	c		e	f	g	a		i		j	k	l	m	n	b

CharThickness INQ

Specifies the maximum thickness (stroke width) of the characters in the OCR font context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the character thickness																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Value >= 1	Specifies the maximum thickness (stroke width) of the characters in the OCR font context, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

StringLength INQ

Specifies the maximum length of the string that can be read or verified using the OCR font context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string length																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
0 <= *Value* <= 100	Specifies the maximum length of the string that can be read or verified, in characters. > Note that if M_SEMI_M12_92 is used, the string length must be 12. If M_SEMI_M13_88 is used, the string length must be 18. The maximum string length is 100 characters.	c		e	f	g	a		i		j	k	l	m	n	b

InitFlag INQ

Specifies whether the characters are brighter than the background. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the character brightness																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FOREGROUND_BLACK	Specifies that the characters to read or verify are darker than the background.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_WHITE	Specifies that the characters to read or verify are brighter than the background.	c		e	f	g	a		i		j	k	l	m	n	b

FontContextOcrIdPtr

Specifies the address of the variable in which to write the OCR font context identifier or specifies the data type that the function should use to return the OCR font context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated OCR font context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated OCR font context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_OCR_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the OCR font context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the OCR font context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated OCR font context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

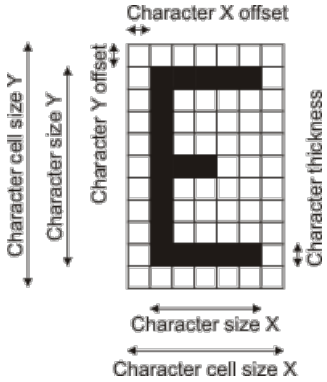
The returned value is the OCR font context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_OCR_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Example

An example of one character from the font, the letter 'E', follows:

```
AIL_ID FontID = M_NULL;
MocrAllocFont(M_DEFAULT, M_USER_DEFINED+M_GENERAL,
              26, 8, 11, 1, 1, 6, 9, 1, 26, M_FOREGROUND_BLACK, &FontID);
```

Would look as follows:



Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrAllocResult

Synopsis

Allocate an OCR result buffer.

Syntax

```
AIL_ID MocrAllocResult(
    AIL_ID SysId,          //in
    AIL_INT64 InitFlag,    //in
    AIL_ID *ResultOcrIdPtr //out
)
```

Description

This function allocates a result buffer for use with other OCR functions. You should check if the operation was successful by verifying that the OCR result buffer identifier returned is not **M_NULL**.

When the result buffer is no longer required, you should release it using the MocrFree() function.

When the OCR result buffer is no longer required, release it using MocrFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the OCR result buffer's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the OCR result buffer.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

InitFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

ResultOcrIdPtr

Specifies the address of the variable in which to write the OCR result buffer identifier or specifies the data type that the function should use to return the OCR result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated OCR result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated OCR result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_OCR_ID* is returned instead of a standard Aurora Imaging Library identifier. This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the OCR result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the OCR result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated OCR result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the OCR result buffer identifier either as a standard identifier (*AIL_ID*) or a smart

identifier (*AIL_UNIQUE_OCR_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrCalibrateFont

Synopsis

Calibrate the OCR font context's character size and target spacing to match the characters in a sample target image.

Syntax

```
void MocrCalibrateFont(
    AIL_ID ImageBufId,              //in
    AIL_ID FontContextOcrId,        //out
    AIL_CONST_TEXT_PTR String,      //in
    AIL_DOUBLE TargetCharSizeXMin,  //in
    AIL_DOUBLE TargetCharSizeXMax,  //in
    AIL_DOUBLE TargetCharSizeXStep, //in
    AIL_DOUBLE TargetCharSizeYMin,  //in
    AIL_DOUBLE TargetCharSizeYMax,  //in
    AIL_DOUBLE TargetCharSizeYStep, //in
    AIL_INT64 Operation             //in
)
```

Description

This function automatically calibrates the X and Y size and spacing of the font of a specified OCR font context to match that of a string in the sample target image. Note that these values can also be set manually with MocrControl() with M_TARGET_CHAR_SPACING, M_TARGET_CHAR_SIZE_X, and M_TARGET_CHAR_SIZE_Y. This function can only calibrate an M_CONSTRAINED OCR font context. You can change the type of context using MocrControl() with M_CONTEXT_CONVERT.

The sample image must contain the string specified in the String parameter, and both the size and spacing must be representative of the images to be read or verified with the calibrated OCR font context. This function might not return the expected results if the sample image contains more or fewer characters than the specified target string length (MocrControl() with M_STRING_CHAR_NUMBER).

The better the sample image, the better the font calibration results. If a string cannot be located in the sample image, an error will be generated. Use MappGetError() to learn the error code and related information.

You can limit the operation to a region of the image buffer, using a rectangular region of interest (ROI) set using MbufSetRegion().

MocrCalibrateFont() should be called after defining a new OCR font context or when the character size in the target images changes.

The resulting target values are the best match between the minimum and the maximum target character sizes specified in the parameters of this function. Font calibration will increment the minimum target character size by a step value until it reaches the maximum target character size. You can inquire the target character sizes and spacing with MocrInquire(). For best results, use a range of +/- 1 pixel with, for example, a step of between 0.1 and 0.2 pixels. Note that the greater the number of steps between the minimum and the maximum target character sizes, the longer this operation will take.

Parameters

ImageBufId

Specifies the identifier of the sample target image buffer. The characters of the string in this sample image must be of the same type and size as that in the images to be read or verified using the OCR font context. The image buffer must be a 1-band buffer.

This image buffer can only have a rectangular region of interest (ROI) defined using MbufSetRegion() (with or without rotation). In addition, the region must be defined in vector format from a 2D graphics list (M_VECTOR or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in raster format (M_RASTER) or a non-rectangular shape.

FontContextOcrId

Specifies the identifier of the OCR font context to be calibrated.

String

Specifies the character string present in the sample target image. This string must be null-terminated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the character string																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*String*") [1]	Specifies the character string that is present in the sample target image. This string must be null-terminated.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

TargetCharSizeXMin

Specifies the minimum width of the characters in the sample target image, in pixels.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the minimum width of the characters																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
6 <= *Value* <= 256	Sets the minimum width, in pixels. Note that the specified value must satisfy the following: (TargetCharSizeXMin / CharSizeX) >= 0.25.	c		e	f	g	a		i		j	k	l	m	n	b

TargetCharSizeXMax

Specifies the maximum width of the characters in the sample target image, in pixels.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the maximum width of the characters																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

6 <= *Value* <= 256	Sets the maximum width, in pixels. Note that the specified value must satisfy the following: (TargetCharSizeXMax / CharSizeX) <= 4.0 .	c		e	f	g	a		i		j	k	l	m	n	b

TargetCharSizeXStep

Specifies the increment to use when searching for the character width between TargetCharSizeXMin and TargetCharSizeXMax. Note that very small step sizes should only be used within small ranges, otherwise application performance can be adversely affected.

TargetCharSizeYMin

Specifies the minimum height of the characters in the sample target image, in pixels.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the minimum height of the characters																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
6 <= *Value* <= 256	Sets the minimum height, in pixels. Note that the specified value must satisfy the following: (TargetCharSizeYMin / CharSizeY) >= 0.25 .	c		e	f	g	a		i		j	k	l	m	n	b

TargetCharSizeYMax

Specifies the maximum height of the characters in the sample target image, in pixels.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the maximum height of the characters																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
6 <= *Value* <= 256	Sets the maximum height, in pixels. Note that the specified value must satisfy the following: (TargetCharSizeYMax / CharSizeY) <= 4.0.	c		e	f	g	a		i		j	k	l	m	n	b

TargetCharSizeYStep

Specifies the increment to use when searching for the character height between TargetCharSizeYMin and TargetCharSizeYMax. Note that very small step sizes should only be used within small ranges, otherwise application performance can be adversely affected.

Operation

Specifies the operation to be performed. Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrControl

Synopsis

Control an OCR font context or an OCR result buffer setting.

Syntax

```
void MocrControl(
    AIL_ID FontContextOrResultOcrId, //out
    AIL_INT64 ControlType,           //in
    AIL_DOUBLE ControlValue          //in
)
```

Description

This function allows you to control an OCR read/verify operation setting. This function also allows you to control various settings that affect how results are retrieved by MocrGetResult(). To inquire the current value of a particular control type, use MocrInquire().

After changing the OCR controls or constraints, use MocrPreprocess() to speed up any following read or verify operation.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MocrControl()** internally calls the *AIL_DOUBLE* version of this function (MocrControlDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MocrControlInt64()).

Parameters

FontContextOrResultOcrId

Specifies the identifier of the OCR font context or the OCR result buffer to control.

ControlType

Specifies the type of control to set.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the required value for the control.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For the characteristics of the font
- For operation controls for read and verify operations
- For the characteristics of the target characters
- For the result buffer

The following ControlType and corresponding ControlValue parameter settings are used to specify the

characteristics of the font.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the characteristics of the font																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_CHAR_ERASE		Erases a character from the font of the OCR font context.															
	Value	Specifies the ASCII character associated with the character. Note that the character must be present in the OCR font context.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to specify operational controls for read and verify operations.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For operation controls for read and verify operations																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_BLANK_CHARACTERS		Sets whether the space between characters in the target image is read or not. Note that this is available only when using an M_GENERAL OCR font context. In addtion, blank characters cannot be verified (using MocrVerifyString()) and will cause an Aurora Imaging Library error. INQ															
	M_DISABLE	Specifies that blank spaces will not be read from the target image into the result string. Note that spaces could still be erroneously read if your OCR font context is not manually calibrated properly or the string length for your target image is wrong. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENABLE	Specifies that blank spaces will be read from the target image into the result string.	c		e	f	g	a		i		j	k	l	m	n	b
M_BROKEN_CHAR		Sets the capability to read/verify a broken character. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that, during the read/verify, OCR should not try to compensate for broken characters. Note that this enables a faster algorithm for reading/verifying a string within the target image. Note that broken characters could still be erroneously read if your target image is noisy, scratched, has low contrast characters, or if the illumination is not uniform. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that broken characters should be identifies as possible characters.	c		e	f	g	a		i		j	k	l	m	n	b

M_CHAR_ACCEPTANCE		Sets the acceptance level used to determine a successful match between the font and the characters found within the target image. During a read/verify operation, the character found with the highest match score, either equal to or greater than the acceptance level, will be returned. If the match score is less than the acceptance level, the result with the highest match score closest to the acceptance level will be returned. If the match score is 0% the invalid character (M_CHAR_INVALID) is returned. A perfect match is 100%, no correlation is 0%. The match score depends on the image quality. You should experiment to decide what is a typical match score for your application. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the acceptance level for a target character. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_INVALID		Sets the ASCII character for all characters within the selected string or strings during the next read or verify operation whose match scores fall below the character acceptance level. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that no special character will replace unrecognized characters. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	1 <= *Value* <= 255	Specifies the character that will replace unrecognized characters.	c		e	f	g	a		i		j	k	l	m	n	b

M_CONTEXT_CONVERT		Changes the type of OCR font context. Note that not all controls are available with both types of OCR font contexts. The restrictions are indicated within each control. When you change the type of OCR font context, invalid controls are replaced by their default value. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONSTRAINED	Specifies an OCR font context that works well with degraded target images. It requires more information about the target string, but provides a more robust search.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GENERAL	Specifies an OCR font context that works well with clean target images. It requires less information about the target string, but provides a less robust search.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRA_CHARACTERS		Sets whether to still read and identify the string when the image contains more characters than otherwise expected. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the read/verify operation should not try to compensate for extra characters.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the read/verify operation should try to compensate for extra characters.	c		e	f	g	a		i		j	k	l	m	n	b

M_MORPHOLOGIC_FILTERING	Sets the number of iterations of morphological filtering. Morphological filtering can add robustness to the OCR operation by internally enhancing the contrast of the image, but it requires processing time. The optimal number of iterations depends on the complexity of the target image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 100	Specifies the number of iterations. 0 disables the morphologic filtering. Only integer values are accepted. The default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_SKIP_STRING_LOCATION	Sets whether to first locate the strings in the target image before trying to identify characters against the OCR font context or to skip this step and save processing time. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the step will not be skipped. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the step will be skipped.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPEED	Sets the algorithm's search speed. Note that increasing the search speed can decrease the robustness and subpixel accuracy of a read/verify operation. This is most effective on large-sized characters. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_HIGH	Specifies a high speed.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOW	Specifies a low speed.	c		e	f	g	a		i		j	k	l	m	n	b

	M_MEDIUM	Specifies a medium speed. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_HIGH	Specifies a very high speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_LOW	Specifies a very low speed.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ACCEPTANCE		Sets the acceptance level used to determine a successful match between the font and a read/verified string. During a read/verify operation, the string found with the highest match score, either equal to or greater than the acceptance level, will be returned. If the match score is less than the acceptance level, the result with the highest match score closest to the acceptance level will be returned. If the match score is 0%, a blank string is returned. Note that, in this case, MocrGetResult() with M_STRING_VALID_FLAG would be false. A perfect match is 100%, no correlation is 0%. The match score depends on the image quality. You should experiment to decide what is a typical match score for your application. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the acceptance level for a string. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE_INTERPOLATION_MODE		Sets the interpolation mode to use when reading/verifying a string at an angle. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_BICUBIC	Specifies bicubic interpolation. The new value is determined by taking a weighted average of the 16 values (4x4) that surround the source point. Note that the sum of the weights used for bicubic interpolation might be greater than one. If this occurs and the result reflects an overflow or underflow, the result is saturated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_BILINEAR	Specifies bilinear interpolation. The new value is determined by taking a weighted average of the 4 values (2x2) that surround the source point. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. The new value is that of the pixel closest to the source point.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOUCHING_CHAR		Sets the capability to read/verify characters that touch each other in the target image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable the identification of touching characters. Enables a faster algorithm for read or verify operations. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable the identification of touching characters.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to set the characteristics of the target characters.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the characteristics of the target characters																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	ControlValue															
M_CHAR_POSITION_VARIATION_X		Sets the amount by which the position of the characters in the target string can vary along the X-axis. This tolerance is relative to the position of each character, as determined by M_TARGET_CHAR_SPACING. This tolerance increases the region being searched for characters within the target image. INQ														
	0.0 <= *Value* <= 100.0	Specifies the position tolerance, in pixels of the character, not of the target image. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_POSITION_VARIATION_Y		Sets the amount by which the position of the characters in the target string can vary along the Y-axis. This tolerance is relative to the position of each character, as determined by M_TARGET_CHAR_SPACING. This tolerance increases the region being searched for characters within the target image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the position tolerance, in pixels of the character, not of the target image. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE		Sets the expected angle at which the string can be found. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_ACCORDING_TO_REGION	Specifies to use the angle of the rectangular ROI (set using MbufSetRegion()) associated with the target image buffer. If the target image is not associated with an ROI, an angle of 0.0 is used. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_ANGLE_DELTA_NEG	Sets the possible angle variation in a clockwise rotation, relative to M_STRING_ANGLE. When searching for a string over a range of angles (setting M_STRING_ANGLE_DELTA_POS and/or M_STRING_ANGLE_DELTA_NEG to a value greater than 1°), the font context's character size must be greater than 16x16 to avoid an Aurora Imaging Library error when calling MocrReadString(), MocrVerifyString() or MocrCalibrateFont(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 180.0	Specifies the possible clockwise angle variation, in degrees. A value less than or equal to 1° will not search over a range of angles. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE_DELTA_POS	Sets the possible angle variation in a counter-clockwise rotation, relative to M_STRING_ANGLE. When searching for a string over a range of angles (setting M_STRING_ANGLE_DELTA_POS and/or M_STRING_ANGLE_DELTA_NEG to a value greater than 1°), the font context's character size must be greater than 16x16 to avoid an Aurora Imaging Library error when calling MocrReadString(), MocrVerifyString() or MocrCalibrateFont(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 180.0	Specifies the possible counter-clockwise angle variation, in degrees. A value less than or equal to 1° will not search over a range of angles. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_CHAR_NUMBER	Sets the length of the string to be read/verified from the target image. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_ANY	Specifies that the length of the string is unknown. Note that this is available only when using an M_GENERAL OCR font context.	c		e	f	g	a		i		j	k	l	m	n	b
	Value <= M_STRING_SIZE_MAX	Specifies the string length to read/verify. The length of this string must be less than or equal to the maximum string length of the OCR font context. Note that when using an M_GENERAL OCR font context and M_BLANK_CHARACTERS are enabled, blank characters must be counted in the string length. The default value is set to the maximum string length of the OCR font context. When OCR font context constraints are set, specifying a string length can improve the speed of a following read/verify operation. Note that if M_SEMI_M12_92 is used, the string length must be 12. If M_SEMI_M13_88 is used, the string length must be 18.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_NUMBER		Sets the number of strings to be read/verified from the target image. Multiple strings can only be found if each string resides on a different line within the target image and each line of text does not overlap the previous. Note that, for best results, all strings read in an image with multiple strings should be of similar length, have a consistent inter-line spacing and should start at a similar location along the X-axis. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies that the number of strings in the target image should be determined automatically.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of lines of text in the target image. Note that specifying this value can improve the speed of a following read/verify operation. The default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b

M_TARGET_CHAR_SIZE_X	Sets the width of the target characters. Note that this can also be automatically set using MocrCalibrateFont(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME	Specifies that the character width is found automatically. Use this value when the font uses a fixed width for all its characters. Note that this is available only when using an M_GENERAL OCR font context.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 1	Specifies the character width in pixels (with subpixel accuracy). The default value is set to the character size X of the font.	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_CHAR_SIZE_Y	Sets the height of the target characters. Note that this can also be automatically set using MocrCalibrateFont(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME	Specifies that the character height is found automatically. Use this value when the font uses a fixed height for all its characters. Note that this is available only when using an M_GENERAL OCR font context.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 1	Specifies the character height in pixels (with subpixel accuracy). The default value is set to the character size Y of the font.	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_CHAR_SPACING	Sets the amount of space between characters in the string. Note that this can also be automatically set using MocrCalibrateFont(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that the inter-character spacing is unknown and not the same between the characters. This enables automatic spacing detection. Note that this is available only when using an M_GENERAL OCR font context.	c		e	f	g	a		i		j	k	l	m	n	b

	M_SAME	Specifies that the inter-character spacing is the same throughout the target string. This enables automatic spacing detection. Note that this is available only when using an M_GENERAL OCR font context.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 2	Specifies the inter-character spacing, in pixels, with subpixel accuracy. The default value is set to the character cell size of the font.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEXT_STRING_SEPARATOR		Sets the ASCII character to be used as a string separator within the text read/verified. This separator will be inserted between each string. Note that this must be set before the text is read/verified. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the ASCII code of the character. The default value is '\n'.	c		e	f	g	a		i		j	k	l	m	n	b
M_THICKEN_CHAR		Sets the number of character thickening iterations. Each iteration enlarges the thickness of the target character. This is useful for some types of fonts (for example, dotted characters). You should choose a value large enough for the intra character dots to connect but not too large to connect separate characters together. Note that the characters included in the font should not be dotted. Instead a thickened version should be included. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 100	Specifies the number of times a character should be thickened. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b

M_THRESHOLD	Sets the threshold value used to internally binarize the target image for segmentation of characters. Note that this control type is available only when using an M_GENERAL OCR font context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to use an automatically computed threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the threshold value. The default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control settings of the result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the result buffer																
ControlType ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RESULT_OUTPUT_UNITS	Sets whether to return results in pixels or world units. This essentially sets the output coordinate system to use. The setting of this control type will only affect functions within this module which return positional results. This control type can be changed at any time to return results in the required output units. INQ															
M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

	M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. If world units are specifed, calling MocrGetResult() generates an error if the result was not calculated on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
M_SELECT_STRING		Selects the line of text to return from the result buffer, when multiple lines of text are read/verified. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies that all strings are selected.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* < M_STRING_NUMBER	Specifies a specific string. Strings are identified by numbers from 0 to (M_STRING_NUMBER-1). The default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void MocrControlInt64 (AIL_ID FontContextOrResultOcrId, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

FontContextOrResultOcrId

See FontContextOrResultOcrId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void MocrControlDouble (AIL_ID FontContextOrResultOcrId, AIL_INT64 ControlType, AIL_DOUBLE
ControlValue)
```

Details

Parameters

FontContextOrResultOcrId

See FontContextOrResultOcrId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrCopyFont

Synopsis

Copy a font character to or from an image buffer.

Syntax

```
void MocrCopyFont(
    AIL_ID ImageBufId,                //in
    AIL_ID FontContextOcrId,          //in
    AIL_INT64 Operation,              //in
    AIL_CONST_TEXT_PTR CharListString //in
)
```

Description

This function copies a character representation of one, or many, font characters to/from the specified image buffer. This buffer can then be used to initialize, change or obtain a visual representation of the font's characters.

If character representations from the image buffer are being copied to the OCR font context, the OCR font context must be large enough to hold the representations of all the specified characters. You can use MocrInquire() to determine the maximum number of characters that can be stored in the font and the size of each character.

The characters are stored in contiguous cells in the image buffer.

The character representations are read from either a font or an image buffer, assuming that the source image is a grid of character representations matching the specified font size. The character representations are read from left to right, and the characters in CharListString must exist in the font.

If the character representation being added already exists in the font, it will be replaced by the new character representation. If the character representation does not already exist in the font, it is appended to the end of the font. If the maximum string length, as set using MocrAllocFont(), is equal to or greater than the number of characters in the font (use MocrInquire() with M_CHAR_NUMBER_IN_FONT), an error will occur and the copy operation will fail.

Note that it is crucial that the character representation respects the foreground value as set in MocrAllocFont(). If the foreground value of the character representation does not match, the font will be unusable.

Parameters

ImageBufId

Specifies the identifier of the image buffer to/from which characters are copied.

This image buffer must be a 1-band buffer and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

FontContextOcrId

Specifies the identifier of the OCR font context to/from which the characters are copied.

Operation

Specifies the direction of the copy operation. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the direction of the copy operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COPY_FROM_FONT +	Copies character(s) from a font context to an image buffer. Note that the maximum number of characters that can be copied from a font context to an image buffer is limited to the number of characters in the font. Note that The buffer height must be at least the character Cell size Y multiplied by the number of lines necessary to hold all the characters. For more details see MocrInquire() with M_CHAR_CELL_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_COPY_TO_FONT +	Copies character(s) from an image buffer to a font context.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for any of the possible values of the Operation parameter.

You can add the following value to the above-mentioned values to specify that all font characters are copied.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the Operations parameter																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALL_CHAR	Copies all font characters to/from the image buffer, stacked from left to right. Note that when M_ALL_CHAR is added, the CharListString parameter should be set to M_NULL and is ignored.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_COPY_FROM_FONT.

You can add the following value to the above-mentioned value to specify that fonts are sorted by their ASCII character values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_COPY_FROM_FONT																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SORT	Sorts font by its ASCII character values.	c		e	f	g	a		i		j	k	l	m	n	b

CharListString INQ

Specifies a string containing the list of characters to be copied.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies to ignore this parameter. This is the only possible setting when copying all characters with M_ALL_CHAR.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*CharListString*") 1	Specifies the string containing the list of characters to be copied. This string must be null-terminated. Character representations copied to the font are associated with the characters, and their ASCII code, in the string. If a character exists in both the list and the font, the character representation will be overwritten with the new character representation. New characters will be added to the font provided that there is sufficient space in the OCR font context. If M_ALL_CHAR is added to the Operation parameter, this setting cannot be used.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrDraw

Synopsis

Draw specific features of OCR fonts or OCR result occurrences in an image buffer or 2D graphics list.

Syntax

```
void MocrDraw(
    AIL_ID ContextGraId,             //in
    AIL_ID FontContextOrResultOcrId, //in
    AIL_ID DstImageBufOrListGraId,   //out
    AIL_INT64 Operation,             //in
    AIL_INT Index,                   //in
    AIL_CONST_TEXT_PTR CharList,     //in
    AIL_INT64 ControlFlag            //in
)
```

Description

This function draws specific features of the OCR fonts or OCR results in the destination image buffer or 2D graphics list.

Parameters

ContextGraId

Specifies the identifier of the 2D graphics context to use when drawing. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. > Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

FontContextOrResultOcrId

Specifies the OCR Font context or result buffer from which to extract the features to draw. The OCR Font context or result buffer must have been previously allocated using MocrAllocFont() or MocrAllocResult().

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or 2D graphics list in which to draw. The buffer can

be any valid 1- or 3-band image buffer allocated using MbufAlloc...(). The 2D graphics list must be previously allocated using MgraAllocList(). By drawing into a display's overlay buffer or associating the 2D graphics list with the display, you can also annotate an image non-destructively.

If specifying an image buffer, it must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Operation

Specifies the type of operation to perform.

The possible Operation parameter values for a OCR Font context are:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of operation to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_CHAR	Draws a character representation of the font in the destination image buffer. Characters are drawn from left to right, and top to bottom. On each line, as many characters as possible will be drawn where the size of each character is the Cell size X. For more information see MocrInquire() with M_CHAR_CELL_SIZE_X. The buffer height must be at least the character Cell size Y multiplied by the number of lines necessary to hold all the characters. For more details see MocrInquire() with M_CHAR_CELL_SIZE_Y. > This operation cannot draw in a 2D graphics list.	c		e	f	g	a		i		j	k	l	m	n	b

The possible Operation parameter values for an OCR result buffer are as follows. These values can be added together to draw multiple features at once.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an OCR Font context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_STRING_BOX	Draws a box around the string that is read in the target. Note that the bounding box of the string is the bounding box of all the string's characters bounding box, including the draw margin.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_STRING_CHAR_BOX	Draws all the character boxes of the result(s) of the string(s) read. The character boxes of the strings are drawn at the location read in the target with the correct angle, scale and aspect ratio. The character box drawn is the bounding box of the character plus the draw margin.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_STRING_CHAR_POSITION	Draws a cross at the center of the bounding box of each string's character. Note that the angle, scale, and aspect ratio of the character are taken into account.	c		e	f	g	a		i		j	k	l	m	n	b

Index

Reserved for future expansion and must be set to **M_DEFAULT**.

CharList

Specifies a string containing a list of valid characters to draw when drawing an OCR font. To draw all the characters in the font or to draw results, this parameter should be set to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*CharList*") [1]	Specifies that, when drawing an OCR font, this parameter can be a null-terminated string specifying a list of valid characters to draw. Characters will be drawn in the specified order. This string must be null-terminated.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrFree

Synopsis

Free an OCR font or result buffer.

Syntax

```
void MocrFree(
    AIL_ID FontContextOrResultOcrId //in
)
```

Description

This function deletes the specified OCR font context or result buffer identifier, and releases any memory associated with it.

All OCR font or result buffers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

FontContextOrResultOcrId

Specifies the identifier of the OCR font context, or result buffer to free. The OCR font context, or result buffer, must have been successfully allocated with MocrAllocFont() or MocrAllocResult(), respectively, prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrGetResult

Synopsis

Get the specified type of result(s) from an OCR result buffer.

Syntax

```
void MocrGetResult(
    AIL_ID ResultOcrId,   //in
    AIL_INT64 ResultType, //in
    void *ResultPtr       //out
)
```

Description

This function retrieves the result(s) of the specified type from an OCR result buffer. The information can be retrieved as a text, a string or a character. A text contains one or more strings. A string contains one or more characters. Results are only available after calling MocrReadString() or MocrVerifyString().

If your target image was associated with a camera calibration context, positional and dimensional results are, by default, returned with respect to the relative coordinate system of the image. Otherwise, these results are returned in pixels, relative to the top-left pixel in the target image.

> If your target image was associated with a camera calibration context but you want to retrieve positional and dimensional results in pixel units, use MocrControl() with the M_RESULT_OUTPUT_UNITS control type set to M_PIXEL. However, note that if you set M_RESULT_OUTPUT_UNITS to M_WORLD without specifying a calibrated image in which to calculate the results, **MocrGetResult**() will generate an error.

Note that, by default, the first string found is returned as a result. To get the result for a specific string or for all strings from a result buffer, use MocrControl() with M_SELECT_STRING.

Parameters

ResultOcrId

Specifies the identifier of the OCR result buffer from which to retrieve results.

ResultType

Specifies the type of result(s) to retrieve. The following values are available to retrieve results from a read or a verify operation, unless otherwise specified.

See the Parameter associations section for possible values that can be specified.

ResultPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]

- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_INT
- AIL_INT16
- AIL_INT32
- AIL_INT64
- AIL_TEXT_CHAR

Specifies the address in which to write the results. Each line of text found within the target image is read as a separate string. If there is only one line of text, there is only one string. Only the results from the first string or selected strings (MocrControl() with M_SELECT_STRING) are obtained.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **ResultPtr** parameter.

- For all strings
- For retrieving one or more strings
- For individual characters

To retrieve a result for all strings, the ResultType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For all strings																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultPtr - Possible values returned																	
M_NB_STRING +	Retrieves the number of strings read by MocrReadString(). If no strings are found, 0 is returned.																
ResultPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TEXT +	Retrieves the entire text. This includes all available strings, their separators, and the terminating null character ("\0"). Note that a string separator is inserted between the different strings when getting all strings simultaneously. The actual string separator can be controlled using MocrControl() with M_TEXT_STRING_SEPARATOR. Refer to M_STRING to retrieve the strings without separators.	c		e	f	g	a		i		j	k	l	m	n	b

ResultPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TEXT_SCORE +	Retrieves the match score for the entire text as determined during the read/verify operation.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD +	Retrieves the value used to binarize the target image. Note that this is available only when using an M_GENERAL OCR font context, or when using an M_CONSTRAINED OCR font context if your target image has multiple lines, or its string angle is not 0.0°.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

The following values are available to retrieve one or more strings, unless otherwise specified. To specify the number of strings to retrieve, use MocrControl() with M_SELECT_STRING. To retrieve a result for a string or all strings, the ResultType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving one or more strings																
ResultType ResultPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_STRING +	Retrieves the null-terminated string(s) found during the read/verify operation. By default, any unrecognized character in the string will be replaced by the most-likely character, even if the match score for that character is lower than the specified character acceptance level (MocrControl() with M_CHAR_ACCEPTANCE). You can replace all unrecognized characters in the string with an invalid character symbol by calling MocrControl() with M_CHAR_INVALID. Note that no string separator is inserted between the different strings when getting all strings simultaneously. Refer to M_TEXT to retrieve the strings with separators.															
ResultPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR*	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_ANGLE +	Retrieves the search angle of the string(s), in degrees, relative to the output coordinate system specified using MocrControl() with M_RESULT_OUTPUT_UNITS. For multiple strings, the search angle of each string is returned. Note that all strings are searched for at the same angle, so all returned values will be equivalent. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_CHAR_NUMBER +	Retrieves the number of characters in the string(s) found during the read/verify operation, not including the terminating null character ("\0") itself; this means it returns the number of characters with results. For multiple strings, the length of each null-terminated string, not including the terminating null character ("\0") itself, is returned.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SCORE +	Retrieves the score calculated during the read/verify operation for the entire string(s). For multiple strings, the calculated score for each string is returned.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_VALID_FLAG +		Retrieves the result of the read/verify operation as a flag that denotes the validity of the string(s). The flag's validity status is determined by comparing the string's read score with the string's acceptance level (MocrControl() with M_STRING_ACCEPTANCE). For multiple strings, a validity flag for each string is returned.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the string's score is below the string acceptance level.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the string's score is above the string acceptance level.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_TEXT; M_STRING.

You can add the following value to the above-mentioned values to get the string length (including the terminating null character).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the string length (including the terminating null character).																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultPtr - Possible values returned																
M_STRING_SIZE +		Retrieves the length of one or more strings, including the terminating null character ("\0"). In the case of M_TEXT, it also includes string separators. This combination constant is useful to establish the required array size to retrieve the string.															
	ResultPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_STRING_CHAR_NUMBER; M_STRING_SIZE.

You can add the following value to the above-mentioned values to specify whether to get the total number of characters for all strings, instead of one value per string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to get the total number of characters for all strings

ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultPtr - Possible values returned																
M_SUM	Specifies to return the total number of characters for all strings, instead of one value per string. Note that the terminating null character ("\0") is ignored by this combination constant. Note that this value cannot be combined with M_TEXT.															
ResultPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

The following values are available to retrieve the results for individual characters in one or more strings, unless otherwise specified. To specify the number of strings to retrieve, use MocrControl() with M_SELECT_STRING. To retrieve a result for the individual characters of one or more strings, the ResultType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For individual characters																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultPtr - Possible values returned																
M_CHAR_POSITION_X +	Retrieves the X-position of each individual character in the string or strings.															
ResultPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_POSITION_Y +	Retrieves the Y-position of each individual character in the string or strings.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SCORE +	Retrieves the match score of each individual character within the string or strings.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SIZE_X +	Retrieves the cell width of each character within the string or strings.	c		e	f	g	a		i		j	k	l	m	n	b

	ResultPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SIZE_Y +		Retrieves the cell height of each character within the string or strings.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SPACING +		Retrieves the spacing between each pair of characters in the selected string,or strings. Note that the spacing of the last character in each string is **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_VALID_FLAG +		Retrieves the result of the read/verify operation as flags that denote the validity of each character within the string or strings. Each flag's validity status is determined by comparing the character's read score with the character's acceptance level (MocrControl() with M_CHAR_ACCEPTANCE).	c		e	f	g	a		i		j	k	l	m	n	b
	ResultPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the character's score is below the character acceptance level.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the character's score is above the character acceptance level.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For retrieving one or more strings, For individual characters. (exceptions might exist)

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																	
ResultType combination value		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultPtr - Possible values returned																

M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
ResultPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For individual characters; and for the following values: M_NB_STRING; M_TEXT_SCORE; M_THRESHOLD; M_STRING_ANGLE; M_STRING_CHAR_NUMBER; M_STRING_SCORE; M_STRING_VALID_FLAG; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested results to a required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
ResultPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr extra info	Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]* **|** *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT16	Casts the requested results to an *AIL_INT16*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr extra info	Data type info: Data type: *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]* **|** *address of an AIL_INT16*	c		e	f	g	a		i		j	k	l	m	n	b

M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_TEXT_CHAR	Cast the requested results to an *AIL_TEXT_CHAR*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr extra info	Data type info: Data type: *address of an AIL_TEXT_CHAR* **|** *array of type AIL_TEXT_CHAR*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrHookFunction

Synopsis

Hook a function to an event.

Syntax

```
AIL_OCR_HOOK_FUNCTION_PTR MocrHookFunction(
    AIL_ID FontContextOcrId,                  //in
    AIL_INT HookType,                         //in
    AIL_OCR_HOOK_FUNCTION_PTR HookHandlerPtr, //in
    void *UserDataPtr                         //in-out
)
```

Description

This function allows you to attach or detach a user-defined function to an event when the specified font is used. A type of event to which a user-defined function can be hooked is string validation. This would immediately follow a read or verify operation. When this type of event occurs, the MocrReadString() or MocrVerifyString() function will call the hooked function one or many times during the operation to validate each string after it is read but before being written to the OCR result buffer. This function allows you to impose global string constraints, and can be used to implement custom checksum functions or to reject strings that would have otherwise met the character constraints imposed.

Note that a function hooked to an event executes on a distinct thread. This permits the functions to run asynchronously from the operation that fired the event and from functions hooked to another event.

Hooked functions should not take longer to execute than the period in which two of their associated events can occur. You cannot determine the instance of the event that fired the function, and even if this were possible, this information would generally not be very useful. Typically, a hooked function performs the minimum number of operations required and, if necessary, performs longer processes by launching other threads.

Parameters

FontContextOcrId

Specifies the identifier of the font.

HookType

Specifies the event type. This parameter can be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the event type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_STRING_VALIDATION +	Specifies the character string to be validated. You must specify a combination value from the following table: • To specify to unhook the event that was hooked to the function	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for any of the possible values of the HookType parameter.

You must add the following value to the above-mentioned value to specify to unhook the event that was hooked to the function.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the HookType parameter to unhook the function																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_UNHOOK	Unhooks the specified function if hooked to the specified event. When you use M_UNHOOK, you must provide the same values for all the parameters as when you originally hooked the function.	c		e	f	g	a		i		j	k	l	m	n	b

HookHandlerPtr

Specifies the address of the function that should be called when the event occurs.

The hook-handler function, pointed to by HookHandlerPtr, must be declared as follows:

```
AIL_INT MFTYPE HookHandler(
    AIL_INT HookType,
    char *StringPtr,
    void *UserDataPtr
)
```

Parameters

HookType

Specifies the type of event hooked.

StringPtr

Points to the string to validate.

UserDataPtr

Specifies the user data pointer passed to **MocrHookFunction**().

The value returned by the hook function must contain the validity status: either **M_TRUE** or **M_FALSE**. Note that, *MFTYPE* and *AIL_OCR_HOOK_FUNCTION_PTR* are reserved Aurora Imaging Library predefined types for functions and data pointers.

UserDataPtr

Specifies the address of the user data that you want to make available to the hook-handler function. This address is passed to the hook-handler function, through its UserDataPtr parameter, when the specified event occurs. Set this parameter to **M_NULL** if the UserDataPtr is not required.

Return value

This function returns a *NULL* pointer.

Remark

- Note that the prototype of this function is kept for backwards-compatibility.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrImportFont

Synopsis

Import character representations into an OCR font context.

Syntax

```
void MocrImportFont(
    AIL_CONST_TEXT_PTR FileName,       //in
    AIL_INT64 FileFormat,              //in
    AIL_INT64 Operation,               //in
    AIL_CONST_TEXT_PTR CharListString, //in
    AIL_ID FontContextOcrId            //out
)
```

Description

This function imports character representations from a file to initialize or overwrite those of an existing OCR font context. This function's main use is to initialize custom fonts.

Font character representations can come from either a grayscale image or a text file.

When taken from an image file, character representations are associated with the ASCII characters in the string, specified by CharListString. The image file must be a grid of character representations matching the specifications of the font. The character representations are read from left to right and top to bottom, and the number of characters in the source image must equal the number of characters in the string, specified by CharListString.

When taken from an ASCII file, font character representations must be presented as in the table below.

```
00 File Representation
01 MIL_ASCII_FONT
02
03 CharValue 65
04 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
06 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
07 00 00 00 00 00 00 00 00 00 00 FF FF FF FF 00 00 00 00 00 00 00 00
08 00 00 00 00 00 00 00 00 00 FF FF FF FF FF 00 00 00 00 00 00 00 00
09 00 00 00 00 00 00 00 00 FF FF FF FF FF FF 00 00 00 00 00 00 00 00
10 00 00 00 00 00 00 00 FF FF FF FF 00 FF FF FF FF 00 00 00 00 00 00
11 00 00 00 00 00 00 FF FF FF FF 00 00 00 FF FF FF FF 00 00 00 00 00
12 00 00 00 00 00 FF FF FF FF 00 00 00 00 00 00 FF FF FF FF 00 00 00
13 00 00 00 00 FF FF FF FF 00 00 00 00 00 00 00 00 FF FF FF FF 00 00
14 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
15 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
16 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
17 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
18 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
19 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
20 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
21 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
22 00 00 00 00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00 00
23 00 00 00 00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00 00
24 00 00 00 00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00 00
25 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
26 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
27 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
28 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
29 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
30 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
31 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
32 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
33 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
34 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
35 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
36 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
37
38 CharValue 66
39 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
40 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
41 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
42 00 00 00 00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00 00 00 00
43 00 00 00 00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00 00 00 00
44 00 00 00 00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00 00 00 00
45 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
46 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
47 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
48 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
49 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
50 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
51 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
52 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
53 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
54 00 00 00 00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00 00
55 00 00 00 00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00 00 00 00
56 00 00 00 00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00 00 00 00
57 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
58 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
59 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
60 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
61 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
62 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
63 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
64 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
65 00 00 00 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 FF FF FF 00 00
66 00 00 00 00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00 00
67 00 00 00 00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00 00
68 00 00 00 00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00 00
69 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
70 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
71 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Row	Description
01	Specifies ASCII file format.
02	Blank row.
03	Specifies the start of a new character representation and its associated (generally ASCII) character.
04 to 36	Specifies the alpha-numerical representation of the character.
37	Blank row.
38	Specifies the start of a new character representation and its associated (generally ASCII) character.
39 to 71	Specifies the alpha-numerical representation of the character.
etc.	This pattern is repeated for every character in the font.

Parameters

FileName

Specifies the name and path of the file from which to import the character representation. The function handles (internally) the opening and closing of the file.

This parameter can be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

AIL_TEXT("*FileName*") [1]	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

FileFormat

Specifies how the data is stored in the file. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how the data is stored																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AIL	Specifies an Aurora Imaging Library format image file. Note that only one-band images can be used.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_ASCII	Specifies a user-drawn ASCII file. Note that the CharListString must be set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIFF	Specifies a TIFF format image file. Note that only one-band images can be used.	c		e	f	g	a		i		j	k	l	m	n	b

Operation

Specifies the type of import operation to be performed. This parameter must be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of import operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_LOAD_CHARACTER	Specifies that the font character representations should be imported.	c		e	f	g	a		i		j	k	l	m	n	b

CharListString INQ

Specifies the ASCII characters to associate with the font character representations in the file. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the ASCII characters

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that this parameter is ignored. Use this setting when the FileFormat parameter is set to M_FONT_ASCII.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*StringOfCharacters*") 1	Specifies the list of ASCII characters to associate with the font character representations in the image file. This string must be null-terminated. Note that the number of characters in this list must match the number of font character representations present in the source file.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

FontContextOcrId

Specifies the identifier of the OCR font context.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrInquire

Synopsis

Inquire about OCR font context or an OCR result buffer setting.

Syntax

```
AIL_INT MocrInquire(
    AIL_ID FontContextOrResultOcrId, //in
    AIL_INT64 InquireType,           //in
    void *UserVarPtr                 //out
)
```

Description

This function inquires about an OCR font context or an OCR result buffer setting.

Note that for an OCR result buffer, this function only retrieves information about result buffer settings (set with MocrControl(), for example). To retrieve results from the OCR result buffer, use MocrGetResult().

Parameters

FontContextOrResultOcrId

Specifies the identifier of the OCR font context or the OCR result buffer from which to read information.

InquireType

Specifies the type of setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT16
- AIL_INT32
- AIL_INT64
- AIL_TEXT_CHAR

- AIL_UINT8

Specifies the address in which to write the requested information. Since the **MocrInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For OCR settings
- For specifying the result for MocrGetResult()
- For inquiring about the system

For an OCR font context, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For OCR settings																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_BLANK_CHARACTERS +		Inquires whether the space between characters in the target image is read. Note that this is available only when using an M_GENERAL OCR font context. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that blank spaces will not be read from the target image into the result string. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that blank spaces will be read from the target image into the result string.	c		e	f	g	a		i		j	k	l	m	n	b
M_BROKEN_CHAR +		Inquires the capacity to read/verify broken characters. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that, during the read/verify, OCR should not try to compensate for broken characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that broken characters should be identifies as possible characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_ACCEPTANCE +		Inquires the acceptance level used to determine a successful match between the font and the characters found within the target image. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the acceptance level for a target character.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_CELL_SIZE_X +		Inquires the font's character cell width. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	6 <= *Value* <= 256	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_CELL_SIZE_Y +		Inquires the font's character cell height. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	6 <= *Value* <= 256	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_IN_FONT +		Inquires the string of ASCII characters associated with the character representations in the font. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b

	M_NULL	Specifies that the parameter is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*StringOfCharacters*") 1	Specifies the string with the list of ASCII characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_INVALID +		Inquires the symbol for the unrecognized characters within the string, or strings. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that no special character will replace unrecognized characters.	c		e	f	g	a		i		j	k	l	m	n	b
	1 <= *Value* <= 255	Specifies the character that will replace unrecognized characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_NUMBER +		Inquires the number of characters that can be stored in the font of the OCR font context. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_NUMBER_IN_FONT +		Inquires the number of characters that are actually stored in the OCR font context.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_OFFSET_X +		Inquires the font's character offset along the X-axis. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the offset along the X-axis, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_OFFSET_Y +	Inquires the font's character offset along the Y-axis. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the offset along the Y-axis, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_POSITION_VARIATION_X +	Inquires the amount by which the position of the characters in the target image can vary along the X-axis. This value is expressed in pixels of the character, not of the target image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the position tolerance, in pixels of the character, not of the target image.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_POSITION_VARIATION_Y +	Inquires the amount by which the position of the characters in the target image can vary along the Y-axis. This value is expressed in pixels of the character, not of the target image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the position tolerance, in pixels of the character, not of the target image.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SIZE_X +	Inquires the width of the font's characters. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	6 <= *Value* <= 256	Specifies the width, in pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SIZE_Y +		Inquires the height of the font's characters. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	6 <= *Value* <= 256	Specifies the height, in pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_THICKNESS +		Inquires the average thickness of the font's characters. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the average thickness, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONSTRAINT + *n* +		Inquires the constraint string for the *n* th position in the read/verified null-terminated string. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that there is no constraint.	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*StringOfCharacters*") 1	Specifies the string with the list of ASCII characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONSTRAINT_TYPE + *n* +		Inquires the constraint type for the *n* th character in the string, as set by MocrSetConstraint(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ANY	Specifies that all characters present in the font are valid.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DIGIT	Specifies that only characters "0" to "9" (ASCII codes 48 to 57) are valid.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LETTER +	Specifies that only characters "A" to "Z" and "a" to "z" (ASCII codes 65 to 90 and 97 to 122) are valid.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT +		Inquires the type of the OCR font context. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONSTRAINED	Specifies an OCR font context that works well with degraded target images. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_GENERAL	Specifies an OCR font context that works well with clean target images. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRA_CHARACTERS +		Inquires whether the string is still identified and read when the image contains more characters than otherwise expected. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the read/verify operation should not try to compensate for extra characters.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the read/verify operation should try to compensate for extra characters.	c		e	f	g	a		i		j	k	l	m	n	b

M_FONT_INIT_FLAG +	Inquires the initialization flag passed at font allocation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_BLACK	Specifies that the characters to read or verify are darker than the background.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_WHITE	Specifies that the characters to read or verify are brighter than the background.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_TYPE +	Inquires the font type of the OCR font context. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SEMI_M12_92	Specifies a font respecting the SEMI M12-92 standard as the type of font. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SEMI_M13_88	Specifies a font respecting the SEMI M13-88 standard as the type of font. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED +	Specifies a general, user-defined type of font. INFO This value is combined with a value from the following table: • To determine the OCR font context type	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE +	Inquires the foreground value of the font. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_FOREGROUND_BLACK	Specifies that the characters to read or verify are darker than the background.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FOREGROUND_WHITE	Specifies that the characters to read or verify are brighter than the background.	c		e	f	g	a		i		j	k	l	m	n	b
M_MORPHOLOGIC_FILTERING +		Inquires the number of iterations of morphological filtering. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 100	Specifies the number of iterations. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESSED +		Inquires whether preprocessing is necessary. The font context must be preprocessed (using MocrPreprocess()) before calling MocrReadString() or MocrVerifyString(). After some of the target constraints or font-specific controls are changed, this inquire type will indicate that the context is no longer in its preprocessed state.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that preprocessing is necessary.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that preprocessing is not necessary.	c		e	f	g	a		i		j	k	l	m	n	b
M_SKIP_STRING_LOCATION +		Inquires whether the string location step will be skipped or not. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the step will not be skipped.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the step will be skipped.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPEED +		Inquires the algorithm's search speed. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_HIGH	Specifies a high speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOW	Specifies a low speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEDIUM	Specifies a medium speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_HIGH	Specifies a very high speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_LOW	Specifies a very low speed.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ACCEPTANCE +		Inquires the acceptance level used to determine a successful match between the font and the read/verified string. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the acceptance level for a string.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE +		Inquires the angle at which the string is expected to be found. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_ACCORDING_TO_REGION	Specifies to use the angle of the rectangular ROI (set using MbufSetRegion()) associated with the target image buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE_DELTA_NEG +		Inquires the possible angle variation in a clockwise rotation, relative to M_STRING_ANGLE. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 180.0	Specifies the possible clockwise angle variation, in degrees. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE_DELTA_POS +		Inquires the possible angle variation in a counter-clockwise rotation, relative to M_STRING_ANGLE. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 180.0	Specifies the possible counter-clockwise angle variation, in degrees. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE_INTERPOLATION_MODE +		Inquires the interpolation mode to use when reading/verifying a string at an angle. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_BICUBIC	Specifies bicubic interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BILINEAR	Specifies bilinear interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_CHAR_NUMBER +		Inquires the length of the string to read/verify from the target image. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ANY	Specifies that the length of the string is unknown. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	Value <= M_STRING_SIZE_MAX	Specifies the string length to read/verify. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_NUMBER +		Inquires the number of strings to read from the target image. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies that the number of strings in the target image should be determined automatically.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of lines of text in the target image. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MAX +		Inquires the maximum string length that will be read/verified using the OCR font context. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the maximum string length.	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_CHAR_SIZE_X +		Inquires the width of the expected target characters. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME	Specifies that the character width is found automatically. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 1	Specifies the character width in pixels (with subpixel accuracy). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_CHAR_SIZE_Y +		Inquires the height of the expected target characters. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME	Specifies that the character height is found automatically. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 1	Specifies the character height in pixels (with subpixel accuracy). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_CHAR_SPACING +		Inquires the expected amount of space between characters in the string. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ANY	Specifies that the inter-character spacing is unknown and not the same between the characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME	Specifies that the inter-character spacing is the same throughout the target string. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 2	Specifies the inter-character spacing, in pixels, with subpixel accuracy. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_TEXT_STRING_SEPARATOR +	Inquires the ASCII code of the character to be used as a string separator within the text read. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the ASCII code of the character.	c		e	f	g	a		i		j	k	l	m	n	b
M_THICKEN_CHAR +	Inquires the number of times a character should be thickened. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 100	Specifies the number of times a character should be thickened. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD +	Inquires the threshold value used to internally binarize the target image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies an automatically computed threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOUCHING_CHAR +	Inquires whether the capability to read characters that touch is enabled or not. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the identification of touching characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENABLE	Specifies to enable the identification of touching characters.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

Combination value for M_CONSTRAINT + n.

You can add the following value to the above-mentioned value to determine the length of the constraint string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the length of the constraint string																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_STRING_SIZE +		Retrieves the length of the constraint string, including the terminating null character ("\0").															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when InquireType=M_CONSTRAINT_TYPE + n: M_LETTER].

One of the following values might be combined with the above-mentioned value to determine the letter case for the constraint.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the letter case for the constraint																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_LOWERCASE	Specifies that only characters "a" to "z" (ASCII codes 97 to 122) are valid.	c		e	f	g	a		i		j	k	l	m	n	b
M_UPPERCASE	Specifies that only characters "A" to "Z" (ASCII codes 65 to 90) are valid.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when InquireType=M_FONT_TYPE: M_USER_DEFINED].

One of the following values is combined with the above-mentioned value to determine the OCR font context type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the context type																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_CONSTRAINED	Specifies an OCR font context that works well with degraded target images and requires more information about the target string, but provides a more robust search. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies an OCR font context that works well with clean target images. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_CHAR_IN_FONT.

You can add the following value to the above-mentioned value to specify that the returned string is sorted in ascending ASCII order.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_CHAR_IN_FONT																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SORT	Sorts the returned string in ascending ASCII order.	c		e	f	g	a		i		j	k	l	m	n	b

For an OCR result buffer, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the result for MocrGetResult()																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_RESULT_OUTPUT_UNITS +	Inquires whether results are returned in pixel or world units. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SELECT_STRING +	Inquires the index of the string, selected from a multiple line text, for which to get the results with MocrGetResult(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies that all strings are selected.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_STRING_NUMBER	Specifies a specific string. INFO	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about the system on which the OCR font context or result buffer is allocated, set the InquireType parameter to the value below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the system																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_OWNER_SYSTEM +	Inquires the identifier of the system on which the OCR font context or result buffer is allocated. SET SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For OCR settings, For specifying the result for MocrGetResult(), For inquiring about the system; (exceptions might exist) and for the following value: M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.																
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested information to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]* **|** *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested information to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]* **|** *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT16	Casts the requested results to an *AIL_INT16*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]* **|** *address of an AIL_INT16*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b

M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_UINT8	Casts the requested information to an *AIL_UINT8*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>]* **|** *address of an AIL_UINT8*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_TEXT_CHAR	Casts the requested information to an *AIL_TEXT_CHAR*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_TEXT_CHAR* **|** *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrModifyFont

Synopsis

Invert or resize the font of an OCR font context to match the target image characters.

Syntax

```
void MocrModifyFont(
    AIL_ID FontContextOcrId, //out
    AIL_INT64 Operation,     //in
    AIL_INT64 OperationFlag  //in
)
```

Description

This function physically modifies the polarity and sizing of the font of an OCR font context.

Parameters

FontContextOcrId

Specifies the identifier of the OCR font context to modify.

Operation

Specifies the type of operation to perform. This parameter can be set to one or more of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INVERT	Physically inverts the character representation of the font (for example, if the foreground is white, this will change it to black).	c		e	f	g	a		i		j	k	l	m	n	b
M_RESIZE	Scales the font. This operation physically changes the size of the characters of the font to match the target character size (M_TARGET_CHAR_SIZE_X and M_TARGET_CHAR_SIZE_Y). Once performed, all of the font's specifications, which were set at allocation (MocrAllocFont()), are modified to this new size. Note that changing the size of the font permanently in the OCR font context can save time when compared to resizing the font before each read/verify operation. Increasing the size of the font might be slower because the resize time might be less than the additional processing time required during a read or verify operation to find characters of a larger font.	c		e	f	g	a		i		j	k	l	m	n	b

OperationFlag

Specifies the interpolation mode for the function. This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the interpolation mode																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_BILINEAR. Note that if the Operation parameter is set to M_INVERT, this parameter should be set to M_DEFAULT. No interpolation is done with M_INVERT.	c		e	f	g	a		i		j	k	l	m	n	b
M_BICUBIC	Specifies bicubic interpolation. The new value is determined by taking a weighted average of the 16 values (4x4) that surround the source point. Note that the sum of the weights used for bicubic interpolation might be greater than one. If this occurs and the result reflects an overflow or underflow, the result is saturated.	c		e	f	g	a		i		j	k	l	m	n	b
M_BILINEAR	Specifies bilinear interpolation. The new value is determined by taking a weighted average of the 4 values (2x2) that surround the source point.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. The new value is that of the pixel closest to the source point.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrPreprocess

Synopsis

Preprocess an OCR font context.

Syntax

```
void MocrPreprocess(
    AIL_ID FontContextOcrId, //in
    AIL_INT64 ControlFlag    //in
)
```

Description

This function preprocesses the specified OCR font context. This function must be called before any read or verify operation and after all the control and constraints are set to speed up all subsequent read or verify operations. Note that preprocessing will be done automatically during the first read or verify operation if this function is not explicitly called.

Each time a font-specific control or target constraint is changed the OCR font context should be preprocessed. To learn if the OCR font context requires preprocessing, call MocrInquire() with M_PREPROCESSED.

Parameters

FontContextOcrId

Specifies the OCR font context identifier.

ControlFlag

Specifies the type of operation to perform. This parameter must be set to the following value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of operation to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Preprocesses the OCR font context.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrReadString

Synopsis

Read an unknown string from a target image.

Syntax

```
void MocrReadString(
    AIL_ID ImageBufId,       //in
    AIL_ID FontContextOcrId, //in
    AIL_ID OcrResultId       //out
)
```

Description

This function reads an unknown string from the specified target image using the specified OCR font context. All existing font controls and constraints (which can be set with MocrControl() and MocrSetConstraint() respectively) are taken into account, and results are stored in the specified result buffer. Results can be read from the result buffer using the MocrGetResult() function.

This function assumes that the string to be read has the same length as specified in the font, and that the target string characters have the same type, size, and spacing as the characters in OCR font context used for font calibration (either automatically with MocrCalibrateFont() or manually with MocrControl()). This is particularly important when using an M_CONSTRAINED OCR font context.

For best results, the angle of the string(s) to be read should be as close to 0 as possible.

You can limit the operation to a region of the image buffer, using a rectangular region of interest (ROI) set using MbufSetRegion().

When reading multiple strings, performing a search at an angle or using the M_GENERAL OCR font context, the target image should have a clearly-defined threshold between the characters and their background. Note that the threshold must preserve the shape of the characters. Broken and/or touching characters can degrade the results.

Note that before performing a read operation, the OCR font context must be preprocessed (MocrPreprocess()).

Parameters

ImageBufId

Specifies the identifier of the image which contains the string to be read. The image buffer must be a 1-band buffer.

This image buffer can only have a rectangular region of interest (ROI) defined using MbufSetRegion() (with or without rotation). In addition, the region must be defined in vector format from a 2D graphics list (M_VECTOR or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in raster format (M_RASTER) or a non-rectangular shape.

FontContextOcrId

Specifies the identifier of the OCR font context to use to read the string from the target image.

OcrResultId

Specifies the OCR result buffer in which to place results of the read operation.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrRestoreFont

Synopsis

Restore an OCR font context, font constraint data, or control data from file.

Syntax

```
AIL_ID MocrRestoreFont(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_INT64 Operation,         //in
    AIL_ID SysId,                //in
    AIL_ID *FontContextOcrIdPtr  //in-out
)
```

Description

This function restores an OCR font context that was previously saved to a file using MocrSaveFont().

This function can also restore a font context from a predefined font context file. Predefined context files are typically distributed with Aurora Imaging Library and can be found in the installed Contexts folder (for example, *C:\Program Files\Aurora Imaging Library\11\Contexts\ Semi_M12-92.mfo*). Alternatively, this function can load only font constraint or control data from the file into the specified OCR font context.

When an OCR font context is no longer required, it should be freed with MocrFree().

After restoring the OCR font context, you should check if the operation was successful, using MappGetError() or by verifying that the OCR font context identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the restored OCR font context is no longer required, release it using MocrFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the restored OCR font context's lifetime and you must not manually free it.

Parameters

FileName

Specifies the path and name of the file from which to restore the Aurora Imaging Library OCR font context (for example, "*C:\mydirectory\myfile*"). The function handles (internally) the opening and closing of the file.

The predefined font files are located in the installed Context folder (for example, "*C:\Program Files\Aurora Imaging Library\11\Contexts*").

To open the file on the remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").

This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Open** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Filename.mfo*") 1	Specifies to restore an OCR user-defined font.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("MICR_CMC_7.mfo") 1	Specifies to restore an OCR font context typically used with the MICR CMC7 standard.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("MICR_E_13B.mfo") 1	Specifies to restore an OCR font context typically used with the MICR E-13b standard.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("OCR_A_BT.mfo") 1	Specifies to restore an OCR font context that uses the OCR-A BT font.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("OCR_B_BT.mfo") 1	Specifies to restore an OCR font context that uses the OCR-B BT font.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("SEMI_M12-92.mfo") 1	Specifies to restore an OCR font context following the SEMI M12 standard.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("SEMI_M13-88.mfo") 1	Specifies to restore an OCR font context following the SEMI M13 standard.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("SEMI.mfo") 1	Specifies to restore an OCR font context typically used with the SEMI conductor standard.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Operation

Specifies which type of OCR font context data to restore. It can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of data to restore																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_RESTORE.	c		e	f	g	a		i		j	k	l	m	n	b

M_LOAD_CONSTRAINT	Loads only font constraint data into the specified OCR font context.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD_CONTROL	Loads only font control data into the specified OCR font context.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE	Allocates a new OCR font context on the specified system, and initializes it with the font, control, and constraint data, restored from file.	c		e	f	g	a		i		j	k	l	m	n	b

SysId

Specifies the system on which the OCR font context is allocated.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

FontContextOcrIdPtr

Specifies the address of the variable in which to write the OCR font context identifier or specifies the data type that the function should use to return the OCR font context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the restored OCR font context (if M_RESTORE was specified); in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the OCR font context (if M_RESTORE was specified); in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_OCR_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the OCR font context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of the Aurora Imaging Library identifier the OCR font context in which to store the loaded data (if M_LOAD_CONSTRAINT or M_LOAD_CONTROL was specified). If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address of the OCR font context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored OCR font context (if M_RESTORE was specified).	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the OCR font context identifier either as a standard identifier (*AIL_ID*) or a smart

identifier (*AIL_UNIQUE_OCR_ID*). If restoration fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrSaveFont

Synopsis

Save an existing OCR font context to disk.

Syntax

```
void MocrSaveFont(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_INT64 Operation,         //in
    AIL_ID FontContextOcrId      //in
)
```

Description

This function saves an existing OCR font context to disk using the Aurora Imaging Library font file format. The OCR font context's control, constraint, and/or font character data can all be saved; which data is saved depends on the value of the Operation parameter.

The font character data contains the character representation of each character in the OCR font context. The control data includes controls used to specify the operational controls for a read/verify operation (such as M_BLANK_CHARACTERS, and M_TOUCHING_CHAR) and those used to set the characteristics of the target characters (such as M_THICKEN_CHAR, and M_TARGET_CHAR_SPACING). The constraint data specifies which characters can appear at given positions in the search string.

Parameters

FileName

Specifies the name and path of the file in which to save the font data. It is recommended that you use the MFO file extension for easier use with other Zebra Aurora software products. The function internally handles the opening and closing of this file. If this file already exists, it will be overwritten.

This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b

AIL_TEXT("*FileName*") [1]	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, OCR font context files have an MFO extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Operation

Specifies which data to save to disk. The Operation parameter can take one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data to save																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Saves the same data as M_SAVE.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE	Saves the OCR font context.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_CONSTRAINT	Saves only character constraint data.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_CONTROL	Saves only control data.	c		e	f	g	a		i		j	k	l	m	n	b

FontContextOcrId

Specifies the OCR font context to be saved.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrSetConstraint

Synopsis

Set character position constraints.

Syntax

```
void MocrSetConstraint(
    AIL_ID FontContextOcrId,           //out
    AIL_INT CharPos,                   //in
    AIL_INT64 CharPosType,             //in
    AIL_CONST_TEXT_PTR CharValidString //in
)
```

Description

This function specifies which character value constraints to apply to each position of the strings in the target image, when using the specified OCR font context. This function specifies which characters can appear at given positions in the string to be read, and associates these constraints with the OCR font context. Using this function increases both speed and reliability as long as the strings in the target image have a known format and obey certain grammatical rules.

Note that constraints are set in regards to the character's position in the string. When dealing with multiple strings, the same constraints are applied to each string.

If you change any constraints, use MocrPreprocess() to speed up any following read or verify operation.

Parameters

FontContextOcrId

Specifies the OCR font context identifier with which to associate the constraints.

CharPos

Specifies the character position in the string for which a constraint is being set. Valid values range from zero to the length of the string -1.

CharPosType INQ

Specifies the type of character that can appear at the specified string position.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of character that is accepted																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that all characters present in the font are valid.	c		e	f	g	a		i		j	k	l	m	n	b

M_DIGIT	Specifies that only characters "0" to "9" (ASCII codes 48 to 57) are valid.	c		e	f	g	a		i		j	k	l	m	n	b
M_LETTER +	Specifies that only characters "A" to "Z" and "a" to "z" (ASCII codes 65 to 90 and 97 to 122) are valid.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_LETTER.

You can add one of the following values to the above-mentioned value to specify the letter case of the characters accepted.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_LETTER to specify which characters are accepted																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_LOWERCASE	Specifies that only characters "a" to "z" (ASCII codes 97 to 122) are valid.	c		e	f	g	a		i		j	k	l	m	n	b
M_UPPERCASE	Specifies that only characters "A" to "Z" (ASCII codes 65 to 90) are valid.	c		e	f	g	a		i		j	k	l	m	n	b

CharValidString INQ

Specifies that any character matching the given CharPosType will be accepted as valid, or explicitly lists valid characters for the specified position.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether all characters matching the CharPosType will be accepted																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Sets all font characters matching the CharPosType parameter value as valid.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*StringOfCharacters*") [1]	Specifies an explicit list of valid characters for the specified position. This string must be null-terminated. The accepted characters must match the CharPosType parameter definition and must exist in the font.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.

DLL	Requires ail11.dll; ailocr11.dll.

MocrStream

Synopsis

Load, restore, or save an OCR font context from/to a file or a memory stream.

Syntax

```
void MocrStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *FontContextOcrIdPtr,   //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load, restore, or save a OCR font context from/to a file or memory stream.

To inquire the number of bytes necessary to save an OCR font context to memory stream, you should first call this function (**MocrStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. This function is equivalent to a file saved using MocrSaveFont().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file. All information about the previously allocated OCR font context is saved, including the OCR font, target image information, and processing controls. However, preprocessing changes are not saved.

All of the OCR font context settings that were in effect when the OCR font context was saved will be restored. A loaded or restored OCR font context is not preprocessed, therefore you must call MocrPreprocess() before performing a read operation with MocrReadString().

Using **MocrStream**(), you can choose to save a backwards-compatible version of the OCR font context, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore OCR font contexts saved using Aurora Imaging Library version 8.0 or above. Settings that do not exist in the lower version will be filled with default values when the OCR font context is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the OCR font context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform on the OCR font context.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the OCR font context. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of stream																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the OCR font context.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

FontContextOcrIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the OCR font context.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the OCR font context, in bytes. If the size is not required, you can set this parameter to **M_NULL**.

> Note that the size of an OCR font context will vary depending on the Aurora Imaging Library version specified.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **FontContextOcrIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																	
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
MemPtrOrFileName - Possible values passed or returned																	
SysId																	
Version																	
FontContextOcrIdPtr - Possible values passed or returned																	
M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save an OCR font context to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.																
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the OCR font context must be compatible when saved. The size of an OCR font context might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b

		M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
	FontContextOcrIdPtr		Specifies the address of the variable containing the identifier of the OCR font context.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD			Loads the content of a specified file or memory stream into a previously allocated OCR font context.	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtrOrFileName		Specifies the file or memory stream from which to load the context.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
		AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. OCR font contexts typically have an MFO file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	FontContextOcrIdPtr		Specifies the address of the variable containing the identifier of the OCR font context. All previous settings are overwritten with those of the loaded context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE			Restores an OCR font context from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtrOrFileName		Specifies the file or memory stream from which to restore the context.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b

	AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. OCR font contexts typically have an MFO file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId		Specifies the system on which to allocate the OCR font context.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
	System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
FontContextOcrIdPtr		Specifies the address of the variable in which to write the identifier of the OCR font context. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE		Saves an OCR font context to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b

	MemPtrOrFileName	Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE For easier use with other Zebra Aurora software products, when saving a OCR font context to a file, use the MFO file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

	Version	Specifies the Aurora Imaging Library version with which the OCR font context must be compatible when saved. Note that the OCR font context can only be up to one major release older than the current version.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
	FontContextOcrIdPtr	Specifies the address of the variable containing the identifier of the OCR font context to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.

MocrVerifyString

Synopsis

Verify a known string in an image.

Syntax

```
void MocrVerifyString(
    AIL_ID ImageBufId,         //in
    AIL_ID FontContextOcrId,   //in
    AIL_CONST_TEXT_PTR String, //in
    AIL_ID OcrResultId         //out
)
```

Description

This function determines whether a known string is present in the target image, and evaluates the quality of the string. This operations can be performed more quickly with **MocrVerifyString**() than with MocrReadString(). After verification, the specified result buffer can be read with the MocrGetResult() function.

> If string location is disabled (using MocrControl() with M_SKIP_STRING_LOCATION), use a child buffer to limit the search area for best results.

The verification can only be performed on a single string and not multiple strings. When dealing with multiple lines of text in a target image, the returned string is the first one found within the target image. Note that this is not necessarily the first string within the target image. The string is located according to its OCR font context type, constraints, and processing controls. Once located, the string can then be read and compared against the validation string provided by this function.

You can limit the operation to a region of the image buffer, using a rectangular region of interest (ROI) set using MbufSetRegion().

For a more robust search, use an M_CONSTRAINED OCR font context. You can change the type of context using MocrControl() with M_CONTEXT_CONVERT.

> Blank characters cannot be verified and will be ignored.

Parameters

ImageBufId

Specifies the identifier of the target image containing the string to be verified. The image buffer must be a 1-band buffer.

This image buffer can only have a rectangular region of interest (ROI) defined using MbufSetRegion() (with or without rotation). In addition, the region must be defined in vector format from a 2D graphics list (M_VECTOR or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in raster format (M_RASTER) or a non-rectangular shape.

FontContextOcrId

Specifies the identifier of the OCR font context to use to verify the string in the target image.

String

Specifies the character string to be verified in the sample target image.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the ASCII characters																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*String*") [1]	Specifies the character string to be verified in the sample target image. This string must be null-terminated. Note that blank characters cannot be verified and will cause an Aurora Imaging Library error.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

OcrResultId

Specifies the OCR result buffer in which to place the verification results.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailocr11.lib.
DLL	Requires ail11.dll; ailocr11.dll.